Evaluation Controls for Architecture Research: Eight Checks That Caught Our Own Wrong Results

A named, reusable protocol of evaluation gates, each derived from a documented wrong result in one of six small-scale architecture projects

Version: 1.0 · **Scope:** small-scale architecture research (≤ 10M parameters; ≤ 5K simulated neurons — in practice tens to hundreds) · **Status of every number in this paper:** quoted verbatim from a named artifact in one of the six source projects, listed in Appendix A.

Abstract

We ran six projects: Paper 1 (matched-cost fractal / weight-tied routing), Paper 2 (the metastable forward map), Paper 3 (scale-recursive routing), the GEODE / DIH-S neuromorphic substrate, the DIH-S v3.0 math audit and experiment ledger, and the HMoE funnel-routing report. Each produced at least one result that was **plausible, internally consistent, reviewed by us, and wrong**. None of those errors was caught by a reviewer, a loss curve, or a sanity check we already had. Every one was caught by a control we then added and named.

This paper documents that protocol. It contributes **no new architecture** and claims none; the architectures under test are reported elsewhere, mostly as negative results. What is contributed here is eight primary controls and two secondary ones, each specified as an enforceable gate with (a) the error class it catches, (b) the case in these projects where it caught a real error that had already produced a plausible-looking positive or negative result, (c) the exact procedure including what to log and what threshold triggers rejection, and (d) the cost measured in wall-clock or experiment count from the projects themselves.

The controls are not novel statistical ideas: held-out validation, matched budgets, null controls and seed averaging are standard practice. What is (possibly) useful is that each one here is stated as a **hard gate with a numeric rejection threshold**, attached to **the specific number it destroyed**, and priced. The protocol exists because every control was written in response to a wrong result that had already survived our initial review — including two we retracted after publishing them internally as headline positives.

The one-sentence summary of the whole document: *a metric you have not gated is a hypothesis, and an architecture claim you have not attacked with a matched control is a story.*

What survives. A reasonable worry about a protocol made entirely of post-mortems is that it is a machine for producing negatives. It is not: it is a machine for producing results that survive, and one of ours did. Paper 2's finding — that a superlinear sublayer reading an un-normalised accumulating residual diverges at finite depth, `RMS(x_{l+1}) ≈ RMS(x_l)·(1 + c·RMS(x_l))` — was established *through* this discipline and then held: five independent causal attacks (init-time divergence with no optimizer step; a 2×2 {shared, per-level} × {norm, no-norm} ablation; fp32 giving no rescue; the init-scale sweep tracking the predicted constant; and NaN incidence falling from 10.34% of logged steps to 0/234), every control green, a mechanism that predicts its own threshold, and a fix whose consequence was measured rather than assumed. That fix also made an entire architecture trainable for the first time — weight tying at 490K parameters went from dead at step 400 to 48.4% exact — which is a positive engineering result, not a negative one. The lesson of C1 is therefore not "your model is broken"; it is that some claims become *more* solid when you force them through the gate, and this paper documents those gates because both outcomes came from the same six projects.

How to read this paper

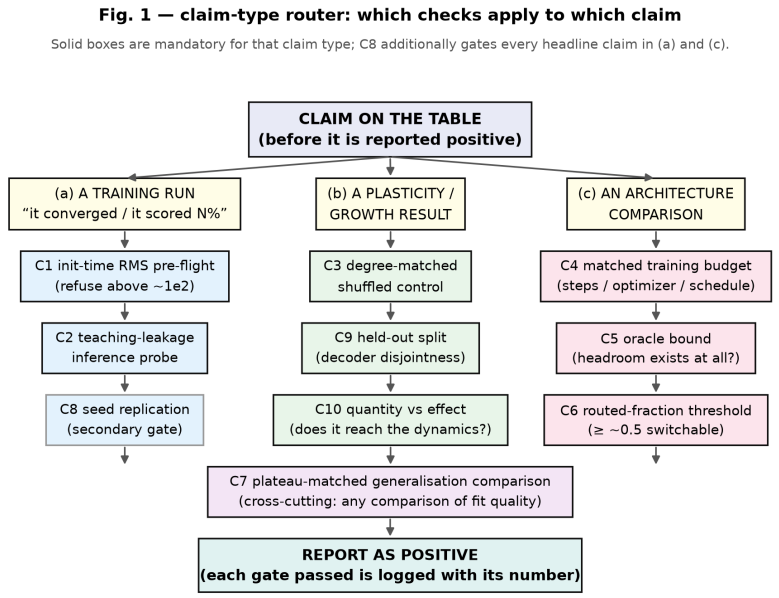
Each control is given in an identical format: **Rule → Error class → Case study → Procedure → Cost → When it fails.** Controls are grouped under four category headings by what they protect: the validity of the forward computation, the integrity of the measurement, the validity of a comparison, and whether the mechanism under study is load-bearing at all.

#	Control	Category	Project that forced it	Figure
C1	Init-time RMS pre-flight	I — forward computation	Paper 2	Fig. 2
C2	Teaching-leakage default hypothesis	II — measurement integrity	DIH-S	Fig. 3
C9	In-sample vs held-out split (secondary)	II — measurement integrity	DIH-S	—
C10	Testing quantities vs testing effects (secondary)	II — measurement integrity	GEODE	—
C4	Matched training budget	III — comparison validity	Paper 1	Fig. 5
C7	Plateau-matched generalisation	III — comparison validity	HMoE	Fig. 8
C8	Seed replication	III — comparison validity	Paper 3	Fig. 9
C3	Degree-matched shuffled control	IV — load-bearingness	GEODE / DIH-S	Fig. 4
C5	Oracle bound as direction-killer	IV — load-bearingness	Paper 1, Paper 3	Fig. 6
C6	Routed-fraction threshold	IV — load-bearingness	HMoE	Fig. 7

Two conventions used throughout, both inherited from the source projects and both themselves controls:

- **Cost is per item, not per token.** Paper 1 §3 records that "assuming a sequence length silently changed conclusions"; MACs/item is MACs/token × the split's own measured mean prompt+target length.
- **An unmeasured claim is not a low-scoring claim.** DIH-S `lab notes/README.md` states the rule for decoders ("any accuracy reported without a stated split is treated as unmeasured, not as a result"), and C2 generalises it. Absent evidence must not be silently converted into a weak positive or a weak negative.

Figure 1 — claim-type router



claim-type router

Fig. 1: which controls must fire, per claim type, before a result may be reported as positive. C8 additionally applies to every headline claim. The protocol is not a checklist to run once at the end; each gate is placed where the corresponding decision is made.

Category I — Validity of the forward computation

This category asks a question that accuracy cannot answer: *is the thing computing at all bounded?* A training run can converge beautifully on top of a forward map that is one perturbation away from arithmetic overflow.

C1 is also where this paper's one durable positive result lives (see "What survives" in the abstract). The control below was written because it caught `D128_long`; the same battery, applied to the mechanism behind it, is what makes that mechanism credible rather than merely plausible. Read the two together: the gate is cheap, and it both kills claims and certifies them.

C1 — The init-time RMS pre-flight

Rule. Log per-level `RMS(x)` at random initialisation with **no optimizer step**, and refuse any configuration whose final-level `RMS(x)` exceeds ~10^2, before spending a single training step on it.

Error class it catches: *silent forward-map divergence* — superlinear sublayer output reading an un-normalised accumulating input, absorbed for thousands of steps by float range and by the optimiser's talent for finding a basin just below the cliff. The general form: **any superlinear sublayer reading an accumulating input is vulnerable**, regardless of architecture family.

Case study (Paper 2 — metastable forward map). `D128_long` reached **59.8% exact match** on character-level arithmetic with no crashes and a converging loss curve (`hmoe/completed/`

`PAPER_2_metastable_forward_map.md` §1, §8; table P2-T1 in `hmoe/research/results/tables.md`). Measured per-level `RMS(x)` on that *trained* checkpoint:

```
level      0     1     2     3     4     5      6       7
RMS(x)   0.1   0.99  1.59  3.15  4.36  6.30   16.35   289.49
```

— **292× growth from level 1 to level 7**, i.e. a forward map that is metastable, not bounded. The same architecture with the fix (`LNE_D128_long`) scored 60.0% at **10.1**× growth; the weight-tied pre-fix variant `STRICT_d128` reached 0.0% and a growth factor of 9.0×10^{13}.

The mechanism was one missing LayerNorm: routed experts read the raw residual stream while the dense comparator LayerNormed its FFN input. Because SwiGLU responds to its input roughly as $\|x\|^2$, each level's update scales with the state it is added to:

```
u_l ≈ c · RMS(x_l)²
RMS(x_{l+1}) ≈ RMS(x_l) · (1 + c · RMS(x_l))
```

a recursion that diverges at finite depth and is *not* an optimisation pathology. It reproduces at random initialisation with no gradient step taken: shared-cell depth 8 without the norm reaches $\mathbf{6.9\times10^4}$, depth 12 reaches `inf → nan`, while the dense control (which has the norm) reaches only **2.22** at depth 8 and **3.46** at depth 16 (P2 §4, table P2-T6). A within-cell measurement isolates it: attention's contribution varies by a factor of 1.10 across levels while the un-normalised expert's varies by 25×, and the implied constants fit `u ~ x²` for the experts (CV 22.7%) and `u ~ x` for the normalised attention path (CV 52.4%) (P2 §3.1). The trigger is routing sharpness, not depth: all eight logged NaNs across 53 pre-fix runs occur at annealed temperature $\geq$ 1.50 (P2 §7.1), and fp32 does not prevent it (fails at step 99 vs bf16's 98).

The lesson, verbatim from the source: *a model reaching high accuracy is not evidence that its forward map is bounded.* Note also what the fix was **not** worth: adding the LayerNorm moved downstream accuracy by only **+0.2 exact / +0.5 answer points** while eliminating every NaN (10.34% of logged steps pre-fix → 0/234 post-fix). A bug can be real, fully explained, and orthogonal to your headline metric simultaneously.

Procedure.

1. Build the model at random initialisation. Load one batch. **Take no optimizer step.**
2. Record `RMS(x)` at every residual level (and along every axis a recursive/nested model has — Paper 3 measures outer *and* inner axes).
3. Log alongside: config hash, depth, width, tied/untied, whether each superlinear sublayer's input is normalised, routing temperature and gate scale.
4. **Rejection threshold: final-level `RMS(x) > 10²` → refuse the configuration.** Healthy profiles in this codebase sit near 1–3; the dense control reaches only 3.46 at depth 16.
5. Sweep the marginal configurations, not just one: over 42 configurations (widths 128/256 × expert widths 64/128/256 × depths 4/8/12 × tied/untied) **12 of 42 already overflow at initialisation**, and survivors span `RMS(x)` from **1.2 to 4.6×10^{12}** — twelve orders of magnitude of behaviour, all of which our logs labelled "it trained."
6. Re-run the gate after any change to sublayer normalisation, init scale, or routing temperature schedule, because `c` in the recursion is set by the expert output-projection init std and the routing gate (`gate_scale · p_sel`).

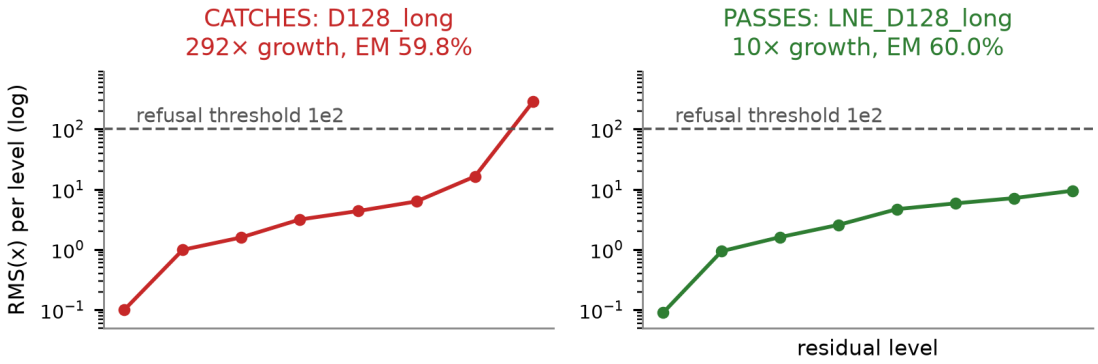
Reference implementation: `scripts/init_blowup.py` (profiles + dense control), `scripts/growth_map.py` (42-config surface), `scripts/ablate_blowup_cause.py` (2×2 {shared, per-level} × {norm, no-norm}), `scripts/`

`temp_coupling.py`, `scripts/trained_residual_profile.py` (`hmoe/completed/PAPER_2_metastable_forward_map.md` Appendix A.1).

Cost to run. One forward pass on one batch: the project's own note is *"~1 min"* for the init profile (P2 §11). The full 42-configuration surface is a single script invocation. Compare the cost of not running it: `STRICT_d128` alone consumed a 3,000-step run that ended permanently dead (`research/STRICT_d128.log`, 913 s), and Paper 1's pre-fix corpus contains 53 runs whose NaN rate was 10.34% of logged steps.

When it fails. If the gate fires, do not re-tune the optimiser — fix the topology: put a normalisation on the superlinear sublayer's **input**, not after its output (the pre-fix code had an `ln4` applied *after* the sum, under a non-default flag, which is why it looked covered). Then verify causality with the 2×2 ablation and confirm that the failure was never what limited accuracy; in Paper 2 the fix was worth +0.2 points of accuracy and 100% of the trainability of one architecture family.

Fig. 2 — C1 init-time RMS pre-flight: 292× vs 10×



C1 paired panels

Fig. 2: C1 catches (292× per-level residual growth on a model scoring 59.8%) versus passes (10× after pre-sublayer normalisation, 60.0%). Both profiles are measured on trained checkpoints; the init-time gate that would have refused the first configuration sees the same split with no training at all — at depth 8 without the norm `RMS(x)` reaches 82.17 against 2.3 with it, and at depth 12 it is `nan`/`inf` against 3.09 (untied) and 7.04 (tied). Both curves stay below float overflow; only one is bounded in the sense that matters.

Category II — Measurement integrity: does the number measure the capability?

This category asks whether the measured variable could have been produced by something other than the capability under study. In DIH-S, five separate leaks and one representability failure all produced accuracy that was real arithmetic on real measurements and meaningless as evidence.

C2 — The teaching-leakage default hypothesis

Rule. Whenever accuracy looks good, the **default hypothesis is that a path still exists by which the training signal reaches the measured variable**; no accuracy number counts until an explicit teaching-off inference probe (washout period + late-trial scoring window) has been run.

Error class it catches: *supervision leakage into the evaluation channel* — including its more dangerous cousin, where removing one leak creates another, and its structural form, where the target is not representable at all so the metric measures an initialisation constant.

Case study (DIH-S / GEODE, the F12 → F21 sequence). The numbers below are from `dihs/lab notes/03-experiment-ledger.md` and `dihs/lab notes/02-math-audit.md`; each is annotated with how it was caught, because that is the part worth stealing.

id	what leaked / failed	the symptom that looked like success	how it was caught
F12	teaching current routed into the readout layer; plateau drive pushed the output soma toward the target	`seq_xor` accuracy **1.000** with no recurrence, on a task whose **memoryless bound from `u_t` alone is 0.57**	accuracy rose monotonically *with position inside a trial* (0.73 → 1.000) — `diag_accuracy.py`. "Accuracy measured in that state is not accuracy — it is the target read back through the apical input."
F13	F12's fix over-corrected: zeroing teaching at the readout also removed its **plasticity** signal	honest XOR stuck at chance (~0.55); only the scalar gain `C` learned; training-length sweep showed accuracy *decreasing* with training length	the VOID training-length sweep (`exp07`, 72 rows) plus `acc_head == acc_tail` everywhere
F16	scoring early in a trial, where the *previous* input is still in the τ_x / membrane traces	`seq_xor` at **1.000** with **no** recurrence	scoring restricted to `(i mod trial) ≥ 0.6·trial` collapsed it → fix `score_from_frac`
F17	VIP/SST states charged by |error| during learning steps persist into the probe window (τ_vip 10 ms, τ_sst 20 ms)	accuracy **1.000 on 3908/3908 scored steps** with all weights frozen except `C`	teaching OFF for the whole probe window + washout → fix `score_last_frac`
F18	teaching current injected into apical **voltage** at ~90 mV/ms against sensory drive 8 mV/ms, so hidden activity during learning *is* the error	accuracy collapses to chance the moment teaching is switched off (XOR → 0.60, seq_xor → 0.47)	`teach_to_voltage=False`: a separate `e_gate` variable carries the error to the plasticity switch only
F21	with one output neuron and non-negative rates, a ±1 gate is **structurally unrepresentable**: `y = tanh(C·x/scale)·scale` with `x ≥ 0` ⇒ `sign(y) == sign(C)` for every input	exactly **0.504 for all 16 configurations × 540 trials** — a frozen sign, not noise	direct measurement: `y` identical for all inputs. "This, not tuning, explains exp10–16 sitting at chance for every configuration."

Two things make this the most instructive failure in the six projects. First, **the leaks were found one at a time and each closure produced a new plausible result**: 1.000 → 0.83 → chance, with a structurally unrepresentable target underneath. Second, after all five leaks were closed the honest answer was that

generalisation to inference is **not demonstrated** — and that negative statement is now gated by an explicit status flag (`hook.py status`, `gate_generalisation`) rather than left implicit.

Honest post-fix numbers for calibration: XOR **0.80–0.83** under the honest probe protocol, best-of-560-run sweep **0.878** (median 0.816), chance 0.50; seq_xor best **0.906**. The 0.92 seen in one diagnostic was itself overfitting (see C9).

Procedure.

```
run(task, probe_every, probe_len, trial_steps,
    score_from_frac >= 0.6, score_last_frac <= 0.4,
    teach_to_voltage = False)
```

1. **Teaching off for the entire probe window**, not attenuated (`teach_to_voltage=False`; a separate variable carries error to the plasticity switch only).
2. **Washout:** discard the first `(1 − score_last_frac) · probe_len` steps of every probe so slow states (inhibitory time constants, traces, normalisers) decay before scoring.
3. **Late-trial scoring only:** count a step if `(i mod trial) ≥ score_from_frac · trial`, with `score_from_frac ≥ 0.6`.
4. **One decision per trial.** Accumulate the decoded output over the scored window and emit a single decision; at R* = 16 Hz a neuron fires ~1.9 spikes in a 120 ms trial, so thresholding a rate variable per 0.25 ms step is meaningless (F20: same network reads **0.424** per-step vs **0.920** trial-averaged).
5. **Bound the task before believing the score.** Compute a memoryless/structural bound on the measured variable from the inputs available at decision time (here 0.57). Any accuracy above the bound is a measurement fault, not a result.
6. **Check representability before tuning.** Verify that the target function is expressible by the readout's output range and polarity (`sign(y)` reachable for both classes); F21 wasted experiments 10–16 on an unrepresentable target.
7. **Log:** every probe parameter above, the bound used, `acc_head` vs `acc_tail` (equality indicates nothing is learning), and accuracy-vs-position-within-trial (monotonicity indicates leakage).

The DIH-S ledger states this as a project-wide rule: *"an accuracy number is meaningless unless produced by `run(..., score_from_frac≥0.6, score_last_frac≤0.4)` with `teach_to_voltage=False`. Numbers from before F18 are not comparable to numbers after it."* Engine defaults now match (`score_from_frac=0.6`, `score_last_frac=0.4`, `teach_to_voltage: bool = False`).

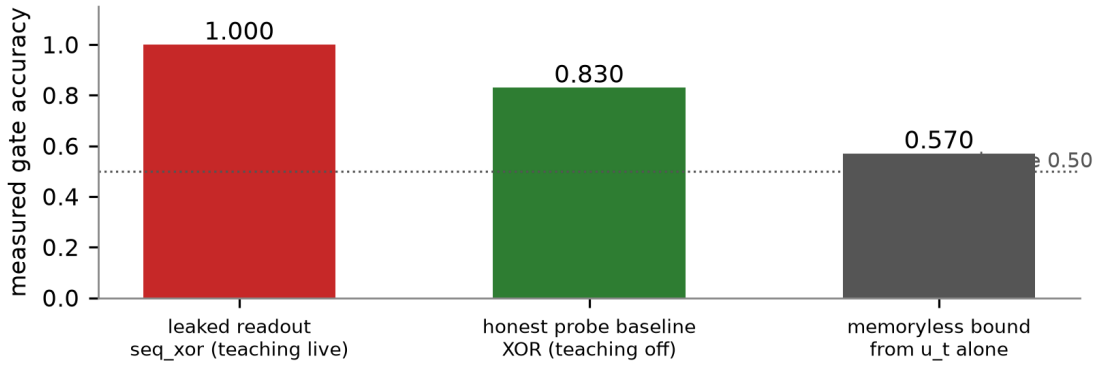
Rejection threshold. Any measured accuracy **above the structural/memoryless bound computed from the decision-time inputs** (here 0.57) is rejected outright as a measurement fault, not reported as a result; and any probe produced without `teach_to_voltage=False`, `score_from_frac ≥ 0.6`, `score_last_frac ≤ 0.4` is rejected on provenance regardless of its value. A target whose polarity/range the readout cannot express (F21) rejects the *configuration*, before any run.

Cost to run. Near-zero per evaluation — the probe is a mode of the existing run loop, and its parameters are engine defaults. The real cost is the re-measurement it forces: exp05 (45 rows) and exp07 (72 rows) were declared **VOID**, and tuning restarted as a 560-run sweep (`exp08`), with 32 runs of inference-robustness (`exp10`) in between. Total ledger cost of the F12→F21 chain: roughly **700+ simulated runs across five experiments**, plus ~127 s for the 30-test verification suite that now locks each fix in.

When it fails. Treat a failure as *unmeasured*, never as low. Then enumerate leak paths mechanically rather than by intuition: every place the target, the error, or any state derived from them can reach (i) the measured neuron's voltage, (ii) its plasticity input in a way that only helps during scoring, (iii) any slow state that outlives the teaching step, and (iv) the readout's representable output set. Close all four,

then re-measure — and expect the number to fall. In DIH-S it fell from 1.000 to chance, which was the correct answer about the architecture and a completely successful outcome for the methodology.

Fig. 3 — C2 teaching leakage: a seq_xor 1.000 that was the target read back (0.83 honest)



C2 paired panels

Fig. 3: C2 — the leaked 1.000 (target read back through the apical input), the honest probe value 0.83, and the memoryless bound 0.57 that made the 1.000 impossible.

C9 — In-sample vs held-out split (secondary control)

Rule. Any decoder must be fitted and scored on **disjoint** trials; an accuracy reported without a stated split is *unmeasured*, not low.

Error class it catches: *decoder overfitting* — the measurement instrument is fitted to the data it then reports on, which is invisible in every per-run log because there is only one dataset involved.

Case study (DIH-S F35, a self-caught retraction). The project's headline positive was that decoding within-trial spike count lifts single-trial accuracy from 0.53 to **0.75–0.78**. `exp33_insample_check.py` re-ran the *identical* decoder with and without a train/test split, 3 seeds × 2 targets (`sim/results/s33_insample_check.csv`; `lab notes/07-gates-and-readout-fixes.md`):

target	in-sample (exp25 method)	held-out	gap
input A	0.683 / 0.687 / 0.683	0.527 / 0.507 / 0.507	+0.157 / +0.180 / +0.177
XOR parity	0.693 / 0.710 / 0.677	0.567 / 0.553 / 0.513	+0.127 / +0.157 / +0.163
mean	**0.689**	**0.529**	**+0.16**

The claim was **withdrawn**: exp24/exp25 had fitted the mean-difference discriminant *and* thresholded at the median using all trials, then scored those same trials. The verdict line in the CSV reads *"CONFIRMED: the 0.75–0.78 was in-sample overfitting."* What still stands is the underlying finding that cross-trial contamination of the readout trace is real (lengthening τ_x monotonically *degrades* decodability, 0.569 at 120 ms → 0.520 at 500 ms), and three independent methods agree held-out single-trial decodability is ~0.51–0.58 — essentially chance.

This produced adopted rule **F36**: *"Every decoder in this project must be fitted on a disjoint trial set, and any accuracy reported without a stated split is treated as unmeasured."* It was caught by re-testing our own positive result, "which is the entire point of keeping an experiment ledger."

Procedure. (1) Split trials — never timepoints within trials. (2) Fit discriminant/threshold/decoder on train trials only, including any median or quantile threshold. (3) Report in-sample and held-out side by side; if only one is reported, the number is unmeasured. (4) Log split sizes and spikes-per-decision available to the decoder — a decision statistic needs enough events to be a statistic (F32: 2 units at ~5 Hz give ~0.7 spikes per 70 ms window, so *any* algorithm decodes chance; budget ≥ ~10 spikes per decision).

Rejection threshold. Any accuracy reported without a stated split is treated as **unmeasured**; and any decoder whose in-sample–held-out gap exceeds ~0.05 (here +0.16) invalidates the *claim*, not just the number — F35 was withdrawn on this basis. Separately, if the decision statistic has fewer than ~10 events per decision (F32: ~0.7 spikes), the task is rejected as unmeasurable before any decoder is fitted.

Cost to run. Essentially free and no new training: exp33 is 6 measurement rows re-scoring an existing decoder across 3 seeds × 2 targets. The cost of not running it was a headline claim that survived long enough to be written into the project's own results section before being retracted.

When it fails. Retract explicitly rather than quietly dropping the number, and keep the surrounding finding if it survives independently (as the τ_x contamination result did). Then treat the gap itself as a quantity worth logging: +0.16 from a two-parameter decoder is a warning about *every* offline decode in the same project.

C10 — Testing quantities versus testing effects (secondary control)

Rule. A test that asserts on a **computed quantity** has not tested that the quantity reaches the dynamics; assert on an observable consequence of the mechanism instead.

Error class it catches: *a feature that exists in configuration and metadata but never acts* — the most expensive class here, because it makes every downstream "the mechanism had no effect" conclusion wrong in direction rather than in magnitude.

Case study (GEODE F40). Conduction delay was a headline variable of the project: geometry-derived axonal delays from per-synapse Euclidean length divided by conduction velocity, quantised into bins. `hist[l]` is allocated with length `d+1` and the newest frame sits at index `d`; the engine read `pre = hist[l-1][d]` — **the current frame**. Every geometry-derived delay was silently zero for the entire project (`dihs/final papers/GEODE-DIH-S-paper.md` §10; `lab notes/07-gates-and-readout-fixes.md`).

The tests passed the whole time: `shape_affects_delays` and `delays_from_geometry` asserted on the computed `delay_steps` numbers, "never whether a delay reached the dynamics — a lesson about testing quantities versus testing effects."

After the fix (pointer ring buffer + `_delayed(l, d)`, with `d = 0` reading exactly one step back, which also removes an algebraic loop), the effect becomes measurable and quantitatively correct: with weights equalised so distance is the only variable, `corr(axon length, first-arrival step)` = **+0.988** / **+0.992** / **+0.984** at 6.0 / 3.0 / 1.5 mm/ms, and arrival spread widens **4.33×** when velocity drops 4× against a theoretical 4× (spread steps 6.0 / 13.0 / 26.0 vs expected 6.1 / 12.1 / 24.2; `sim/results/s40_delay_isolated.csv`). The regression test is named for the effect, not the quantity: `conduction_delay_reaches_dynamics`.

The same family of bug appears twice more in this project group, which is why we treat it as a control rather than an anecdote: F23 (pruning silently disabled by an `if self.W_rec:` nesting — mask density stayed at exactly 0.625 for every prune threshold from 1.2σ to 4σ while 754 hidden spikes occurred) and F41 (a drive-budget mismatch producing literally **zero** downstream spikes, which initially read as "delays don't work").

Procedure. For every mechanism you introduce:

1. Name an **observable that only the mechanism can produce**, chosen so its magnitude is predicted by theory (here: arrival-step spread should scale linearly with 1/velocity).
2. Isolate it: equalise everything else (weights, drive, counts) so the mechanism's variable is the only one moving — `exp40_delay_isolated.py`.
3. Assert on the observable **and** on its predicted scaling, not on the intermediate configuration value. A test that passes with the mechanism stubbed out is not a test.
4. Add a negative control at the same time (bidirectional wiring: spikes returning to layer 0 after the direct volley = **60 with bidirectional wiring, 0 without**).
5. Log which quantity each existing test asserts on. Any test whose assertion target is a config-derived number gets flagged for an effect-level rewrite.

Rejection threshold. If stubbing out the mechanism leaves the test passing, the test is rejected as a config assertion and rewritten to assert on the observable; if the measured observable does not scale with the mechanism's parameter as theory predicts (delay-spread scaling, here corr **+0.988** after the plumbing fix versus no dependence before it), the mechanism is declared **inert** and every downstream conclusion that assumed it is void.

Cost to run. One isolated experiment plus one regression test; the DIH-S suite runs 30 unique tests in ~127 s (max per-test times from `lab notes/test-log.csv`, 240 logged entries). The cost of not having it was measured differently: an entire project phase of geometry/delay results computed against a mechanism that contributed exactly zero.

When it fails. Fix the plumbing, then **re-run every downstream conclusion that assumed the mechanism was active**, and expect direction changes rather than magnitude changes. In this case the honest statement became "delays demonstrably shape arrival statistics; no task here yet shows a behavioural dependence on them" — which is a different claim, not a weaker one.

Category III — Validity of a comparison

This category protects against comparing two things that are not comparable. It is where our most confident architecture-level conclusions were wrong, in both directions.

C4 — The matched-training-budget rule

Rule. No architecture comparison is valid unless both arms are trained for the same number of steps with the same optimizer and schedule, stated per row.

Error class it catches: *baseline undertraining* — reading a gap as an architectural property when it is a property of the training schedule. It produces false optimism about your own architecture, which is the kind of error you never find unless you look for it, because it feels like progress.

Case study (Paper 1, self-correction against our own report). Earlier drafts of `hmoe/research/FINDINGS_AND_THEORY.md` celebrated `D128_long` at **59.8% exact and 66.4% below** the dense L8F128 reference. Retraining the *same architecture and hyperparameters* at a longer budget (`research/results/tables.md`, table P1-T5):

dense L8 ffn128, calc_mix	steps	exact	answer	MACs/item
`dense_calc_mix_L8_F128`	3,000	61.0%	61.8%	127,777,183
`REF9k_L8F128`	9,000	**73.2%**	**74.1%**	127,777,183

+12.2 exact points from training length alone. The consequence is stated in the source without hedging: *"Every 'matched quality' statement in this project made before this control existed was comparing a 3k-step dense model against fractal models trained for 9k steps. The gap I was closing did not exist; the bar moved when I measured it properly."* The lesson line: *"a compute saving is only meaningful against a baseline trained under the same budget. It is trivially easy to beat an undertrained dense model with a well-trained anything, and I nearly did exactly that in a report."*

Two secondary confounds were caught in the same sweep and belong to this control: **checkpoint-selection noise** (`D128_long`'s best-by-exact checkpoint is 59.8%/60.6% at step 9000; best-by-answer is 59.2%/60.7% at step 8000 — a 0.6-point spread, which must be applied identically to both families), and **dataset/trace-length mismatch** (dense trained on 44-token items versus fractal on 30-token items is not a comparison; measured emitted length was identical across architectures at 17.3 tokens, so that saving belongs to the task, not the architecture).

Procedure.

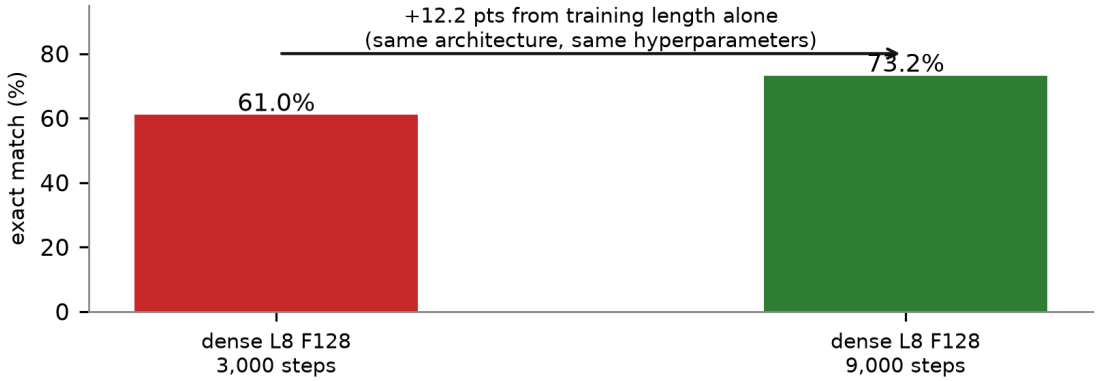
1. Fix one optimizer and schedule per comparison family, and state **steps, optimizer, LR schedule, dataset name, and checkpoint-selection rule in every row of every table**. A row without a budget is a row without a meaning.
2. Run both arms at *the same* budget; if you also report a longer budget for one arm, report the same arm's shorter-budget number next to it (that is exactly what P1-T5 does).
3. Sweep the baseline as hard as you sweep your architecture — "equal tuning effort" is the same control seen from the other side. HMoE §12.2 implements it mechanically: one identical LR × seed grid applied to every arm, compared within schedule with **equal cell counts**, and each architecture shape scored against dense separately rather than pooling shapes.
4. Log the budget as a first-class column in the results store so that any downstream report must inherit it (HMoE's `experiments.csv` carries `steps` and `wall_secs` per run; its report generator refuses hand-typed numbers).

Rejection threshold. A comparison is rejected if the two arms differ in steps, optimizer state, LR schedule, or tuning-cell count, or if any row lacks a budget column. Operationally: **if retraining only the baseline at the proposed arm's budget moves its metric by more than the claimed margin (here 12.2 points vs margins of 4–8), the comparison is void** and must be re-run before any claim is written.

Cost to run. Cheap, and this is the point: the control that invalidated a headline claim was **one retraining run of an existing baseline at a longer budget — 263 s wall-clock** (`hmoe/research/REF9k.log`, `REF9k_L8F128` 9,000 steps, RTX 3080). The comparators are similarly cheap: the five dense frontier runs finish in **144–163 s** each; the 3.75M-parameter fractal `D128_long` takes **1,130 s**. Equal-tuning-effort grids cost cells, not discoveries: HMoE ran **50 tuning cells totalling 2.94 h** of wall-clock across both arms (from `funnel/experiments.csv`).

When it fails. Re-baseline everything and re-derive the verdict; expect verdicts to flip in *sign*, not shrink. In Paper 1 the corrected comparison turned "66% cheaper at matched quality" into a Pareto-dominance result against our own architecture (5 of 5 dominance tests, `PAPER_1_matched_cost.md` §4). Record the pre-correction claim in the log rather than editing it out — that record is C4's evidence base.

Fig. 5 — C4 matched training budget: the baseline that moved 12.2 points under us



C4 paired panels

Fig. 5: C4 — the same dense architecture at two training budgets. The 12.2-point difference is a property of the schedule, and it was being read as a property of our architecture.

C7 — The plateau-matched generalisation comparison

Rule. Compare two architectures only after restricting to cells from each arm whose final training loss is below a strict threshold (e.g. 0.02), so fit is matched **by construction**, with no interpolation.

Error class it catches: *fit mistaken for learning* — and its dual, "capacity ceiling" claims made about a model that never failed to fit. Two architectures can differ on held-out data while both being at zero training loss, in which case every capacity-based explanation is false and the difference is one of generalisation.

Case study (HMoE funnel report §12.4–12.6). Both arms reached near-zero training loss (dense best 0.0005, routed 0.0010 — a 2.1× ratio on quantities that are both ≈ 0), and **both memorised the training split**: best train-split exact match **0.993** (routed) vs **0.998** (dense). Their held-out scores were **0.461** and **0.653** — gaps of +53.2 and +34.5 points on the same checkpoints.

Restricting to plateau cells (final training loss < 0.02) removes the confound entirely:

arm	plateau cells	test EM (mean)	min	max	worst train loss in set
sparse routed	5	**0.451**	0.438	0.461	0.0040
dense matched	7	**0.565**	0.491	0.653	0.0188

+11.4 EM points of generalisation gap, which the report calls "the cleanest number in the report" because it is not explained by capacity (both fit perfectly), by tuning (each arm's own plateau cells) or by interpolation (none performed).

The control also caught its own weaker variant. An earlier interpolated version of the same comparison — matching each arm's loss→EM curve at a common training loss — gave only +2.07 points at loss 0.05, and the report explicitly refuses to prefer it: interpolation walks through cells trained at *different learning rates*, averaging "same loss, different hyperparameter" pairs, which flattens the difference rather than removing confounds. Choosing between a matched-by-construction comparison and an interpolated one is itself a methodological decision that changes the answer by 5×.

Procedure.

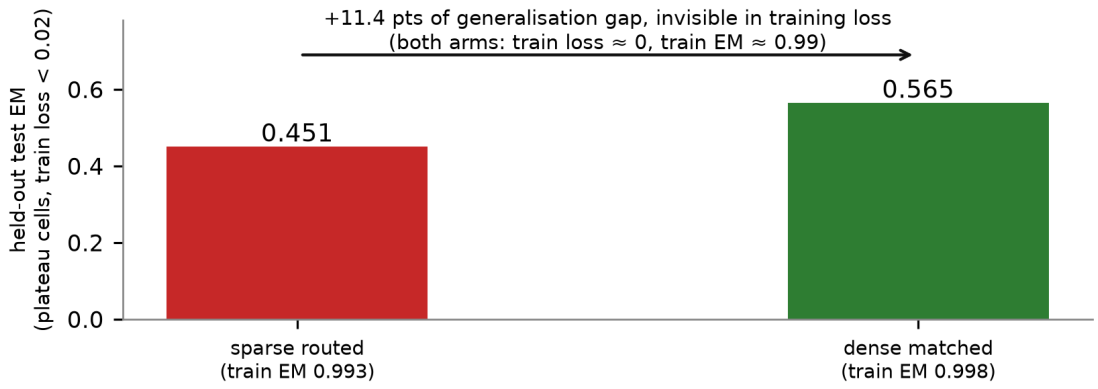
1. Log **final training loss per cell** for every arm — including dense. (If your results store has no train-loss column, add it before comparing anything.)
2. Choose a threshold defining "has fit the data" (here 0.02; report the worst train loss inside each selected set so the reader can see the matching is real).
3. Restrict both arms to their plateau cells and compare held-out metrics on that subset only. Report cell counts per arm; if one arm has no plateau cells, that is a trainability result, not a generalisation one.
4. Exclude diverged cells (here EM < 0.1) with the reason logged: *"a divergence is not a measurement of capacity."*
5. Also report the **train-split score of the same checkpoints**, which puts fit and generalisation on one scale and makes memorisation visible.
6. If you additionally interpolate, label it indicative and never let it replace the matched-by-construction number.

Rejection threshold. A generalisation comparison is rejected if either arm's cells sit outside the fit plateau (**final training loss > 0.02**, with the worst train loss inside each selected set reported — here 0.0040 routed / 0.0188 dense), or if fewer than 4 plateau cells exist per arm (the study has 5 and 7), or if diverged cells (EM < 0.1) remain in the set. Only after both arms have *fit* may held-out numbers be compared at all.

Cost to run. Zero training: this control is a filter over runs you already have (HMoE's comparison used 5 + 7 existing cells; analysis in `funnel/hmoe/generalize.py`). Its real cost is architectural — you must have logged training loss and train-split scores per cell, which projects that only log validation accuracy discover they cannot run it at all.

When it fails. A large plateau-matched gap says your architectures differ in *transfer*, not capacity; stop arguing about parameter counts and start looking at optimisation dynamics and inductive bias. Note the budget dependence the same section records: going 5,000 → 12,000 steps moved dense +11.1 points and routed +3.5 points, so "routing costs N points" must be read as "under this optimizer budget." C7 without C4 is a measurement of the schedule.

Fig. 8 — C7 plateau-matched comparison: fit matched by construction, generalisation is not



C7 paired panels

Fig. 8: C7 — held-out EM at matched fit (train loss < 0.02, train EM 0.993 vs 0.998): 0.451 routed vs 0.565 dense. Training loss cannot see this gap; neither can train accuracy.

C8 — The seed-replication requirement

Rule. Single-seed results are not results: every headline claim must replicate at a second seed, and the honest claim is the intersection of the seeds, not the union.

Error class it catches: *seed noise presented as an effect* — including the subtlest form in these projects, where an advantage that replicates in **direction at one granularity rung** fails to define a *scaling* claim, because the ladder of rungs is itself seed-dependent.

Case study (Paper 3 — scale-recursive routing). Three headline claims were rerun at seed 1 with identical configurations (`completed/PAPER_3_scale_recursive_routing.md`, T11 and §7 verdict):

comparison	seed 0	seed 1
nested vs coarse flat, 18k-step budget	**+6.4 points** (p = 0.0011)	**−1.8** (p = 0.18)
fixed-nested vs fixed-flat (routers replaced by fixed random subsets), G=8	**+3.4** (p = 0.01)	**−5.4** (p = 3.1×10^{-5})
nested vs coarse flat, G=8 (tie claim)	−0.6 (p = 0.72)	−4.6 (p = 2.3×10^{-5})

The source states the consequence plainly: *"the correct claim is therefore narrower than the one this section opened with,"* and the surviving claims are exactly those that replicated — e.g. nesting at matched granularity **+8.2 points at seed 0 (61.00% vs 52.80%, p = 1.1×10^{-8}) and +3.0 at seed 1**, with the fine-granularity rung also replicating (+8.0 / +9.6). Note that even a replicated advantage can be non-load-bearing: replacing every sub-router in the nested model with a **fixed random subset average** (bit-identical parameter count and MACs/item) made it *better* — val 61.60% vs 58.60%, paired test 66.00% vs 61.00%, discordant 32/7, McNemar p = 1.2×10^{-4} — and the same substitution applied to the **flat** family gained *more* (+3.2 and +9.8 val points), making fixed-flat the best configuration in the study at both seeds (64.8% at seed 0; 65.8% at seed 1 vs fixed-nested 60.4%, p = 3.1×10^{-5}). The source's own conclusion: *"Fractal routing does not carry the benefit; neither does hierarchy — routing is the cost."* That is C6's conclusion arriving by another route, and it is why C8 and C6 must be run together: a replicated advantage can still belong to the wrong mechanism.

Procedure.

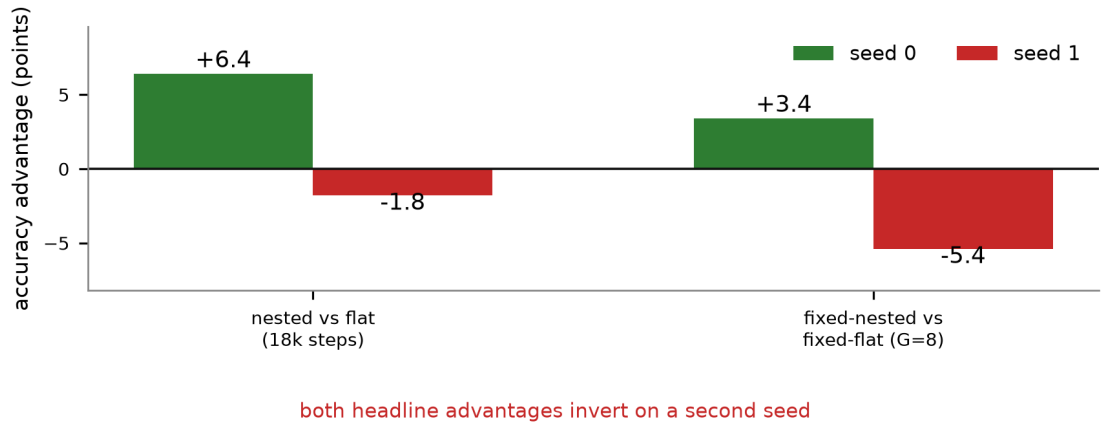
1. Rerun **every headline comparison**, not a random sample, at ≥ 1 additional seed with all other configuration fields byte-identical (config hash in the log).
2. Report per-seed values side by side plus a paired test on shared items (Paper 3 uses McNemar on 500–1,000 shared items; e.g. p = 0.72 at seed 0 vs p = 2.3×10^{-5} at seed 1).
3. **Claim the intersection.** If a comparison flips sign, the claim becomes "no reproducible advantage," not "advantage with variance."
4. For scaling/budget-ladder claims, require replication **at every rung**: monotonicity across rungs is itself a single-seed observation. Paper 3's budget ladder (G=8 → G=16 → 18k steps) inverted at seed 1 and therefore cannot support a scaling statement.
5. Log the number of seeds per claim as a column in the results store, so a report generator can refuse to print a headline with n_seeds = 1.

Rejection threshold. A headline claim with `n_seeds = 1` is rejected (not "reported with variance"), and a claim whose sign flips between seeds is rewritten to **"no reproducible advantage"** — the intersection, never the average. Concretely: +6.4 → −1.8 and +3.4 → −5.4 across two comparisons both trigger rejection; so does a budget ladder that fails to replicate at any rung.

Cost to run. The measured cost is the honest price of this control: **9 seed-1 replicate runs totalling 50,857 s ≈ 14.1 GPU-hours** (`hmoe/research/FR*_s1.log`; individual runs 356–10,301 s). Roughly one extra run per headline comparison; in Paper 3 the replicate wave changed the paper's conclusion, so the alternative was publishing a result that reversed.

When it fails. Narrow the claim, in writing, to what replicates — and check whether the failure is *variance* or *mechanism*: at seed 1, "nesting loses to coarse flat" and "removing the router helps" were both true, which together say the router was never the source of the advantage. A failed replication is a result about your mechanism, not just about noise.

Fig. 9 — C8 seed replication: two seed-0 wins that reversed at seed 1



C8 paired panels

Fig. 9: C8 — two seed-0 advantages (+6.4 and +3.4 points) that invert at seed 1 (-1.8 and -5.4).

Category IV — Is the mechanism load-bearing at all?

This category asks the question architecture research most reliably avoids: if we remove the thing we are studying, what changes? Three of these controls were built after a mechanism was found to be decorative, non-specific, or incapable by construction.

C3 — The degree-matched shuffled control

Rule. No plasticity or growth result is believed without a **conductance-normalised, degree-matched shuffled control** that grows the same number of synapses by the same rule from shuffled temporal order; if the control does at least as well, there is no specific effect to report.

Error class it catches: *non-specific gain presented as learned structure* — any intervention that adds capacity or excitation inflates correlation-style and lock-style measures, so an uncontrolled "success" is the expected outcome of a real measurement with the wrong comparison.

Case study (DIH-S F34, three attempts, two invalid). The goal was to grow structured connectivity from an observed brainwave template. Attempt 1 (`exp28`) was **invalid**: with all-to-all connectivity every parcel mixes every input channel, so the observed inter-parcel phase step was **0.5°** — a global flash, not a travelling wave — and "before growth" scored 0.846 against almost anything, making the comparison meaningless ("`growth_helped = False` only by luck"). Attempt 2 (`exp28b`) fixed the stimulus but not the comparison: with local connectivity (radius 3 mm) the network oscillated at **3.33 Hz** against a 2 Hz drive (band fraction 0.394; phase spread still 0.24 rad), and growing **930** template-ordered lead→follow

synapses raised replay phase-lock from **−0.791 to +0.209**, "which looks like success." It is not: the shuffled control rose further (**+0.367**).

Attempt 3 (`exp28c`) added both fixes — rescale after growth so total conductance equals the un-grown baseline (weights × **0.46**), and grow a second, degree-matched treatment from a **shuffled** template, each scored against its own template (`sim/results/s28c_template.csv`):

template	growth	lock-to-own-template	activity	new synapses	w-scale
true	none	−0.827	1.978	0	—
true	grown	**−0.498**	1.546	447	0.460
shuffled	none	−0.832	1.978	0	—
shuffled	grown	**−0.057**	1.599	462	0.451

Gain from growth: **+0.329 for the true template vs +0.775 for the shuffled control.** The matched control beats the treatment ⇒ `specific_effect = 0`. Interpretation from the source: adding ~900 recurrent synapses mainly raises gain, which increases any oscillation's amplitude and therefore *any* phase-lock measure. Adopted rule: *"no plasticity or growth result is accepted unless a degree-matched, conductance-normalised control fails to do at least as well"* (also codified as methodology rule 2 in `lab notes/README.md`). The paper adds the honest diagnostic location: the failure sits in the growth protocol (a static template applied all at once) and in weak entrainment (3.33 Hz endogenous vs 2 Hz driven), not in the premise that templates can shape structure.

The same artefact class recurs later in the project's credit-assignment work: *"the F34 lesson (a measure that rises with any activity is worthless) applied to credit"*, and as methodology rule 3 — a "credit degrades accuracy from 1.00 to 0.50" result evaporated when the identical arm was run with credit *off*, because `eta = 0` with `alpha, gamma > 0` dissolves the hidden layer regardless of the mechanism under test.

Procedure.

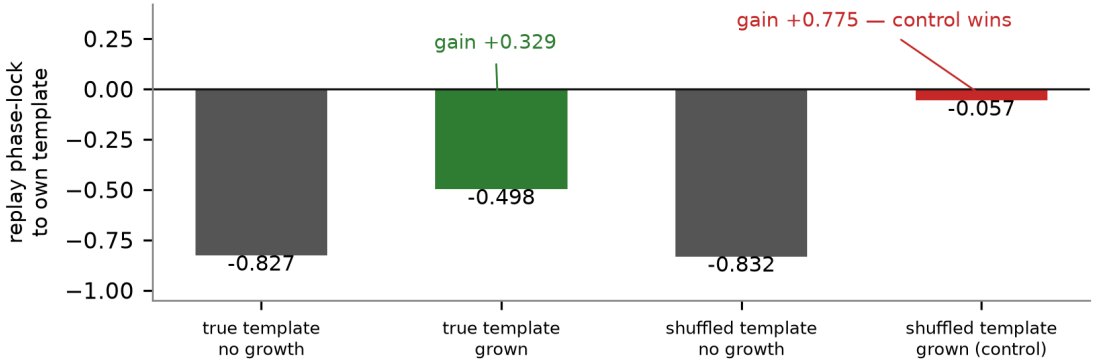
1. Grow the treatment: N new synapses by rule R from the true temporal order.
2. **Conductance-normalise post-growth**: rescale weights so total conductance equals the un-grown baseline, and log the scale factor (here 0.460 / 0.451).
3. Grow the control: same N (matched to within a few synapses — 447 vs 462), same rule, same normalisation, from **shuffled** temporal order; score each condition against its own template.
4. Compare **lock-to-own-template between conditions**, not absolute improvement within the treatment arm.
5. Also log: total activity per condition (a gain mechanism predicts amplitude changes — here 1.978 → 1.546/1.599), new-synapse counts, and a no-growth baseline for each template.
6. **Rejection threshold: control gain ≥ treatment gain ⇒ no specific effect.** Report `specific_effect = treatment_gain − control_gain` as the headline quantity, never the treatment gain alone.

Cost to run. Two conditions instead of one — roughly 2× per growth experiment, and here three iterations (exp28, exp28b, exp28c) before the design was even valid; `s28c_template.csv` carries four condition rows plus three summary lines (`true_gain`, `shuffled_gain_matched_control`, `specific_effect`) for the final verdict. The cost of skipping it: two experiments that "would both have been publishable as 'structure grew from an observed brainwave template'."

When it fails. Report the negative and locate the failure precisely (protocol vs premise), then fix the protocol — incremental Hebbian expression while plasticity is open, and actual entrainment — rather

than re-tuning the measure. Note that failing this control is cheap information: it told us our phase-lock metric was gain-sensitive, which generalised to the credit-attribute work immediately.

Fig. 4 — C3 degree-matched shuffled control: growth gain +0.329 vs +0.775 shuffled



C3 paired panels

Fig. 4: C3 — growth "improves" phase-lock under both templates; the shuffled control improves it more (+0.775 vs +0.329), so the effect is non-specific excitation.

C5 — The oracle bound as direction-killer

Rule. Before investing in a mechanism, compute the **strongest possible version of it** and compare to the target: if the oracle cannot reach the bar, no learned policy can, at any policy quality.

Error class it catches: *engineering toward a headroom that does not exist* — the mechanism is implemented, trained, measured, and reported against a baseline that made its ceiling look attainable.

Case study 1 (Paper 1 — adaptive depth has zero headroom). The forced-depth curve of `D128_long` (n=500, exact match per difficulty bucket at exactly *k* executed levels; `research/FINDINGS_AND_THEORY.md` §3, oracle block in `research/results/tables.md`) shows genuine depth heterogeneity — trivial saturates by depth 4 while easy is still improving at 6 — which is exactly the pattern that motivates a depth router. Then the oracle: perfect per-bucket depth choice given the model's own measured accuracy-vs-depth curve.

- best **fixed** depth: **61.87%** at depth 6
- **oracle** maximum accuracy: **61.87%** at depths (trivial, easy, medium, hard) = (3, 6, 6, 6)
- **oracle gain over the best fixed depth: +0.00 points**
- accuracy monotone non-decreasing in depth: **False** — extra depth actively hurts

Adaptive-depth routing cannot recover a gap that a perfect depth oracle cannot even see. The same section records the temptation it killed: measured against the *undertrained* dense baseline (61.0%, see C4), the oracle showed 79–85% compute savings at matched accuracy — "exactly the kind of number that would have gone into a white paper unchallenged, and it is real arithmetic on real measurements."

The caveat that makes this control honest. Look at the oracle's chosen depths: `hard = 2`. It harvests savings by dumping the hard bucket to minimum depth **because that bucket scores 0–3% at every depth anyway** (hard-bucket exact accuracy by depth: [0.0, 0.0, 0.0, 0.0085, 0.0169, 0.0085]). The source's wording is the rule: *"Routing away compute from items you will get wrong regardless is real accounting but it is not intelligence … an 85% 'saving' assembled that way measures how hopeless the hard items are,*

not how good the router is.” Therefore: always report per-bucket accuracies alongside any oracle saving, and never quote a min-cost saving whose chosen depths dump a bucket.

Case study 2 (Paper 3 — same verdict at every granularity rung). Rebuilt for nested routing (`completed/PAPER_3_scale_recursive_routing.md` T4), the oracle reproduces the conclusion structurally: with accuracy monotone in outer depth, each ceiling *is* its full-depth point and **every bucket’s best depth is full depth**, so there is no per-bucket depth headroom anywhere; min-cost targets are reached “only by dumping the medium/hard buckets to depth 1 — the accounting artefact flagged in FINDINGS §3.” Nesting raised the fixed-depth model’s ceiling instead of creating routing value. (The cross-family ceiling gaps at G=16 — 63.66% vs 56.08% — and at 18k steps — 72.27% vs 68.49% — are seed-0 numbers that inherit the seed-1 reversal reported in C8, which is why they are not claimed here.)

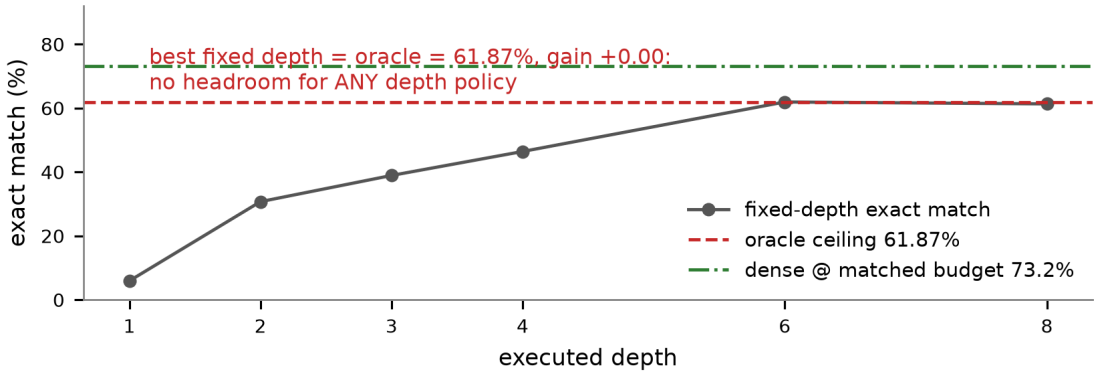
Procedure.

1. `forward_fixed(k)`: evaluate the trained model at each forced capacity/depth `k`, per difficulty bucket, on held-out items. Log the full curve, and explicitly check monotonicity in `k` (non-monotonicity is itself a finding).
2. Compute the **oracle**: best-per-bucket choice subject to minimum expected cost for a target accuracy. Report (i) oracle maximum accuracy, (ii) best fixed choice, (iii) **oracle gain = (i) − (ii)**, (iv) the chosen depths/paths per bucket, (v) each bucket’s accuracy curve.
3. Compute min-cost-per-target-accuracy over the oracle’s choices, and mark targets `UNREACHABLE` rather than extrapolating (here 65% and 70% are unreachable at a ceiling of 61.9%).
4. **Rejection rule: if oracle gain ≈ 0 (or the oracle cannot reach the properly trained baseline), stop building the learned policy.** No policy beats an oracle that has nothing to exploit.
5. Inspect *how* any oracle saving is obtained; discount savings assembled by dumping hopeless buckets.

Cost to run. One forced-depth evaluation sweep per model: Paper 1’s oracle used n=500 held-out items over 6 depth settings (`scripts/difficulty_depth.py`), Paper 3’s covered four difficulty buckets × depth settings for the seven named comparison models (`scripts/oracle_hier.py`, `scripts/hier_difficulty_curve.py`). No training required — this control runs on checkpoints you already have, which is what makes running it *before* building the mechanism so cheap.

When it fails. The direction is dead; report it as a structural negative with the oracle numbers, and redirect to where the ceiling actually moves (Paper 3: raising the fixed-depth model’s ceiling, not routing within it). Also re-check the baseline first — an oracle that looks spectacular is usually measured against an undertrained comparator (C4).

Fig. 6 — C5 oracle bound: a perfect per-bucket depth oracle gains exactly nothing



C5 paired panels

Fig. 6: C5 — a perfect per-bucket depth oracle matches the best fixed depth exactly (61.87% vs 61.87%, gain +0.00), while the properly trained dense baseline sits at 73.2%.

C6 — The routed-fraction threshold

Rule. Routing is only load-bearing above roughly **50% of parameters being switchable**; below that, test the router by ablation and expect to find it decorative.

Error class it catches: *decorative mechanism* — a router trained, tuned, instrumented, and reported as working in a model where at most 8–27% of parameters can be switched off, so almost nothing is riding on its decision.

Case study (HMoE funnel report §11). 59 runs were re-evaluated with the routing decision replaced (uniform / random / fixed-path), holding weights, tile capacity and decoding identical, then split by measured **routed fraction** = parameters routing can switch ÷ total parameters (threshold 0.50):

- **22 low-routed runs (max routed fraction 0.273):** the worst decision-preserving ablation costs at most **8.65 EM points**, and **9 of them improve** when routing is replaced with random or fixed-path selection.
- **37 high-routed runs (min routed fraction 0.684):** the same ablations cost **1.53–14.27 EM points** across the 26 runs that did not collapse (EM $\geq$ 0.1) — i.e. **23 of those 26 sit in a 3–15 point band**, with only three below 3 points — and **none of those improves**.

The report's verdict: *"Routing therefore only carries signal once the decision actually controls the model's capacity; where tiles are redundant, the router is decorative."* The structural cause is arithmetic and was visible from parameter counting alone before any training: in the original shared-trunk design, tiles were **830,080 of 9,607,384** parameters — the other 8,777,304 are dense trunk capacity no gate can bypass — so selecting 8 of 10 tiles leaves only **1.73%** of parameters inactive per token and the routed model's active count at 0.947× the dense baseline's total. Moving FFN capacity into routed tiles (trunk = attention only) raises routed fraction to 0.769–0.825 and makes 14.23% of parameters inactive per token; that is when ablations start costing points.

Denominator discipline (a control on this control). Counting all 37 high-routed runs includes cells whose training collapsed (EM $\approx$ 0.03), where removing routing cannot hurt because nothing is working; the report's own summary sentence counts those in and reads "the worst costs −0.03 points and 7 gain from removal." Restricting to non-collapsed cells gives 26 runs, 1.53–14.27 points, none improving — the same conclusion with an honest denominator. Report which set you are quoting; a threshold rule computed over an undisclosed mixture is not reproducible.

Procedure.

1. Compute and log **routed fraction** = **switchable parameters** / **total parameters** for every run, at config time, before training. In HMoE this is `_routed_frac()` in `funnel/hmoe/report_matched.py` over the tile/trunk parameter split maintained by `hmoe/active.py` ("only expert tiles are switchable; the trunk (embeddings, …) is not"), and the 0.50 cut is an explicit constant with its provenance recorded in code: `ROUTED_SPLIT = 0.5 # data-derived split of routed fraction` (`funnel/hmoe/report_routing_load.py`).
2. If routed fraction < ~0.50: do not report the router as a contribution. Run the ablation battery anyway and expect it to be flat.
3. Re-evaluate the trained checkpoint with the routing decision replaced by (a) uniform averaging, (b) random selection, (c) a fixed path. **Verdict on the worst selection-preserving probe** (`random`,

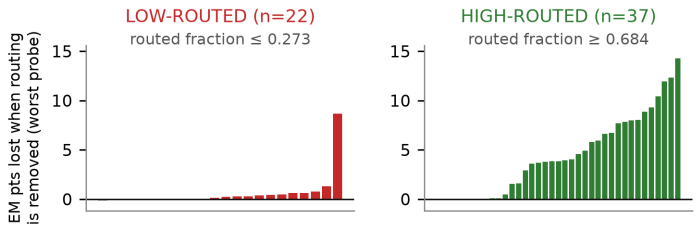
`fixed_path`) only — `uniform` changes effective capacity as well as the choice, so it cannot separate "wrong choice" from "more compute."

4. Log per run: learned EM, each ablated EM, each drop in EM points, and `min decision drop`.
5. **Rejection thresholds:** routed fraction < 0.50 ⇒ mechanism out of scope for a sparsity/efficiency claim; `min decision drop` ≤ noise band (HMoE's measured seed-to-seed spread was 0.010 EM) ⇒ router decorative at this budget, independent of routed fraction.
6. Report the two panels separately (low- vs high-routed), never pooled.

Cost to run. Near-zero for the threshold itself (a parameter count computed before training). The ablation battery is evaluation-only over existing checkpoints: three probes × 59 runs, backfilled by `funnel/hmoe/backfill_routing_ablations.py`, which **defaults to dry-run and writes only with `--apply`**. For scale, HMoE's entire 80-run experiment corpus cost **5.0 h** of wall-clock (`experiments.csv`, median run 261.5 s), so the ablation pass is a fraction of one training campaign.

When it fails. If routed fraction is low, fix the architecture (move switchable capacity into the routed units) rather than the router — that is exactly what HMoE's `fix.md` demanded and what the E8/E10 sparse arms did. If routed fraction is high but ablation cost is ≤ noise, the router is decorative at that budget and the honest claim is about capacity organisation, not adaptivity.

Fig. 7 — C6 routed-fraction threshold: the router is decorative below ~0.5 switchable parameters



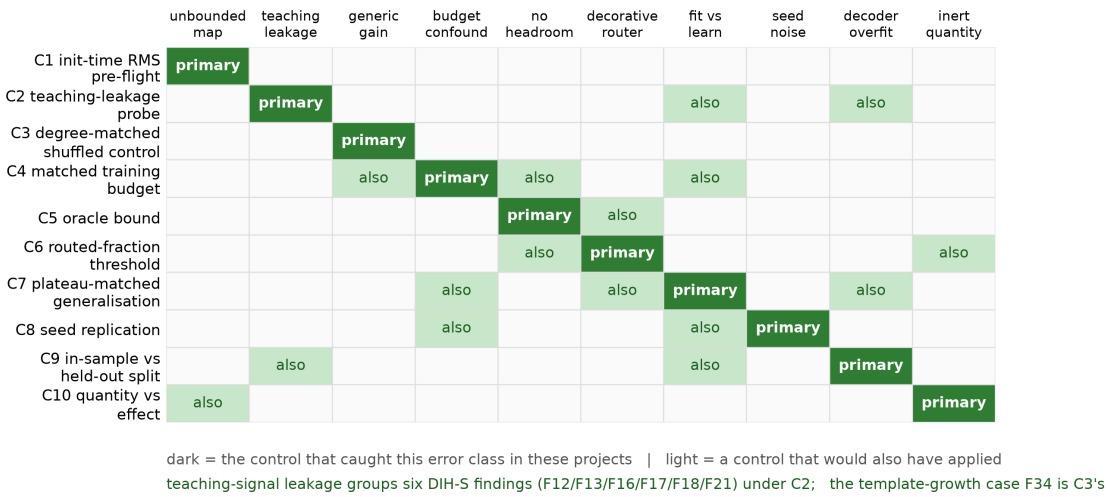
LOW panel: worst cost 8.65 pts and 9 of 22 IMPROVE when routing is removed. HIGH panel: converged runs lose 1.53–14.27 pts; none improves.

C6 paired panels

Fig. 7: C6 — per-run cost of removing the routing decision. Low-routed runs (≤0.273 switchable parameters) cluster at zero with 9 of 22 improving when routing is removed; converged high-routed runs (≥0.684) lose 1.53–14.27 EM points.

Figure 10 — check × error-class matrix

Fig. 10 — check × error-class matrix: which control catches which failure

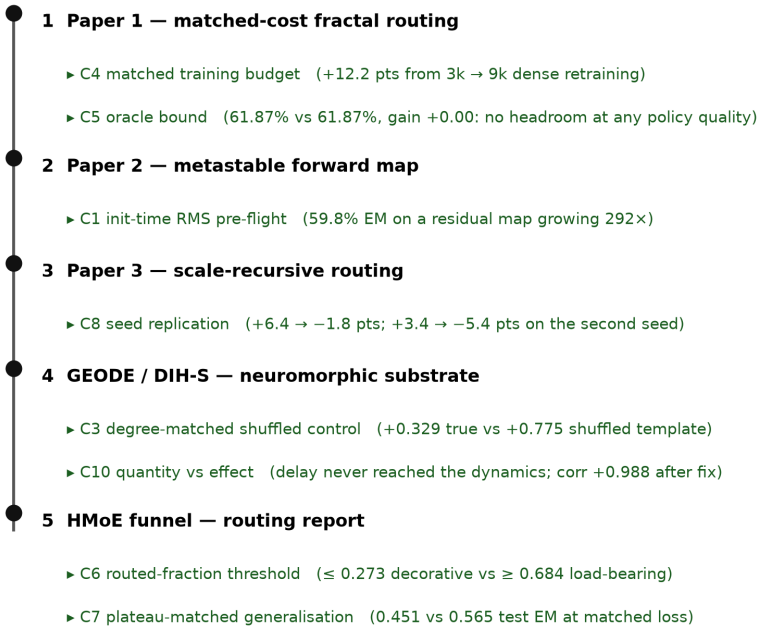


check × error-class matrix

Fig. 10: which control catches which error class in these projects (dark = the control that actually caught it there; light = a control that would also have applied). Column tags expand to: unbounded forward map; teaching-signal leakage; non-specific gain; baseline undertraining; no headroom in the mechanism; decorative router; fit mistaken for learning; seed noise; decoder overfitting; quantity never reaches the dynamics. The DIH-S leak sequence F12/F13/F16/F17/F18/F21 groups under C2; the template-growth case F34 is C3's.

Figure 11 — development timeline

Fig. 11 — development timeline: every control exists because a specific wrong result survived review first



development timeline

Fig. 11: which check was added in response to which project, in order: Paper 1 → matched budget (and the oracle bound); Paper 2 → init-time RMS; Paper 3 → seed replication; GEODE/DIH-S → degree-matched control and quantity-vs-effect; HMoE → routed-fraction threshold and plateau-matched generalisation. C9 arrived with F35/F36 in the same DIH-S phase.

How to apply this in an agentic research loop

These controls are not review advice; they are **the tests an agent should run against its own outputs before it reports a result as positive.** Every one of them exists because a wrong result had already survived our initial review — including two that were written up as headline positives and later retracted (F34, F35). An autonomous research loop does not reduce this risk on its own: it produces plausible-looking results faster, reviews them less, and has no instinct about which number is too good. The gates below are placed where an agent makes decisions.

Gate 0 — before any training run (C1). Emit per-level `RMS(x)` at init with no optimizer step; refuse above 10^2. Cost: one forward pass. An agent that skips this spends GPU time discovering arithmetic and will misattribute it to optimisation, exactly as our own logs did ("bf16 overflow", written into the trainer's NaN guard, was the wrong mechanism).

Gate 1 — before reporting any accuracy (C2, C9, C10). Confirm the measurement channel: teaching/supervision off, washout applied, late-window scoring, one decision per trial, a structural bound computed and not exceeded, decoders fitted on disjoint trials, and — for any mechanism claim — an assertion that the mechanism reaches the dynamics rather than that its configuration value was set. If any of these is missing, output **"unmeasured"**, never a number. This is the single highest-value rule in the protocol: it converts "we don't know" into an explicit state instead of a weak positive.

Gate 2 — before reporting any comparison (C4, C7, C8). Both arms at equal steps/optimizer/schedule with the budget printed per row; equal tuning cells per arm; fit matched by construction before generalisation is discussed; ≥ 2 seeds for every headline claim, with the claim narrowed to the intersection. An agent should refuse to emit a comparative sentence whose rows lack a budget column — this is mechanically checkable in a report generator, and HMoE's `tests/verify_report.py` is an existence proof that "no unsourced numbers" can be enforced by machine rather than by care.

Gate 3 — before claiming a mechanism does something (C3, C5, C6). Run the matched control (shuffled/null/degree-matched/conductance-normalised), run the oracle to check headroom exists at all, and compute the switchable fraction of parameters so the mechanism's reach is known before it is credited. The agent's default conclusion when the control matches the treatment is **no effect**, and that conclusion should be written up with the same detail as a positive — in these six projects negative results "carried most of the information."

Loop hygiene. Three properties made the controls work here, and all three are implementable by an agent: (i) every run has an id, a config record, and a log directory, so a control can be re-run against old artefacts rather than only against new runs (HMoE backfills ablations into pre-existing `summary.json` files; Paper 3's measurement pass takes *every* run id including replicates and controls); (ii) results are generated from the store, never typed, so a control that changes a number propagates automatically; (iii) the ledger records VOID results and retractions, because C4's entire evidence base is a claim we made and then disproved. A loop without an append-only record of its own wrong answers cannot accumulate methodology — it can only accumulate results.

One caution specific to agents. These gates fire often and slow things down; an agent optimised for producing positive results will be tempted to treat them as advisory. Two properties of the evidence

above should settle that: the errors they catch were not extreme (a 1.000 accuracy, a +6.4-point win, a 59.8%-accurate model with a diverging map), and each had already been accepted by a competent reviewer — us.

Limitations

Scale. Every project behind this protocol ran at or below **10M parameters** (Paper 1's corpus: 481,920–8,531,081 parameters; Paper 2 states its range as 0.5–9M; Paper 3's matched families are ~0.7M–3.7M with a hard 10M cap in HMoE) or at the scale of **tens to a few hundred simulated spiking neurons** (DIH-S/GEODE configurations: [4,32,1] and [4,64,1] populations, hidden layers of 64–128 units, an 8→64→64→24 distance-wired mesh — far below 5K neurons). Nothing here is measured at scale.

Our negative results may not scale. "Weight tying buys parameters, not compute"; "adaptive depth has no headroom on this task family"; "routing costs ~11 EM points of generalisation at equal size"; "template-driven growth produces non-specific gain" — each is stated in its source with the task and budget attached, and each could reverse at larger scale, longer training, different data, or higher precision. Paper 3's own reciprocal controls show how quickly sign flips inside a fixed budget (C8), which should make us *more* cautious about extrapolating across two orders of magnitude of scale, not less. Paper 2 states the analogous limit for its mechanism: demonstrated for SwiGLU experts reading an un-normalised residual; the argument only uses $\|u(x)\| \sim \|x\|^2$, so we *expect* other superlinear sublayers to behave the same but did not test them.

Thresholds are calibrated, not derived. The 10^2 init-RMS refusal is set against healthy profiles of 1–3 in one codebase; the ~50% routed-fraction threshold is where HMoE's own bimodal distribution splits (max 0.273 vs min 0.684 — nothing in between, which is why that cut is clean there and may not be elsewhere); `score_from_frac ≥ 0.6` / `score_last_frac ≤ 0.4` come from τ_x = 25 ms against trial lengths in one spiking engine; the 0.02 plateau threshold is a round number that happened to separate "has memorised" from "has not." Port them as defaults and re-calibrate on your own healthy baselines.

Coverage limits. These controls address *measurement* validity, not scientific questions: they cannot tell you whether an architecture is worth building, whether a benchmark matters, or whether your metric is the right one — only whether the number you are about to report means what you think. Three of them can be satisfied by a model that memorises (C7 exists precisely because C1/C2/C4 were all green on models whose train EM was 0.99). Two projects record further controls in the same family that we did not promote here because they came from single cases: run-the-null-in-the-same-cell, log-the-weight-norm-not-just-accuracy, and measure the informativeness (d′) of the learning signal itself (`dihs/lab notes/README.md`, rules 3–5).

Not a universal methodology. This is a set of safeguards for small-scale architecture research under agentic or semi-autonomous operation, derived from the dozen-odd documented failures catalogued above, all in six projects. It is not a claim that architecture research generally is broken, not a substitute for statistical design, and not novel statistics: held-out validation, matched budgets, null controls, oracle bounds and seed averaging are standard practice whose status we upgraded — from "good hygiene" to "hard gate with a numeric rejection threshold and a logged case where it fired."

Provenance. Every number, quotation, and case study in this paper traces to a named artifact inside the six project folders (Appendix A), and two scripts re-check that claim mechanically: one matches 91 quoted case-study strings against the file each is cited to, the other recomputes the aggregate cost figures from run logs and result stores. Where a figure could not be sourced it was removed rather than estimated — which is why every cost quoted here is a wall-clock or experiment count taken from a log, not an approximation.

Appendix A — provenance index

Every number in this paper traces to one of the following. Where we cite a script, it is named as the source project names it. Case-study metrics are quoted verbatim from these records; cost figures that are aggregates (wall-clock totals, run counts, table row counts) were recomputed for this document from the artifacts named in them — `REF9k.log`, `D128_long.log`, `FR*_s1.log`, `funnel/experiments.csv`, `lab notes/test-log.csv` and the §11 ablation table — by `audit_provenance.py`, which is included alongside this file and prints each recomputation.

Control	Primary artifacts
C1	`hmoe/completed/PAPER_2_metastable_forward_map.md` (§1, §3.1, §4, §5, §7.1, §8, §9, §11); `hmoe/research/results/tables.md` (P2-T1…T8)
C2	`dihs/lab notes/03-experiment-ledger.md` (exp05 VOID; F16/F17/F18 table; protocol rule; post-fix honest baseline; exp08/exp10); `dihs/lab notes/02-math-audit.md` (F12, F20, F21); `dihs/sim/dih/engine.py` (probe parameters), `dihs/sim/dih/params.py:158`; `dihs/final papers/GEODE-DIH-S-paper.md` §4.6–4.7
C3	`dihs/sim/results/s28c_template.csv`; `dihs/lab notes/07-gates-and-readout-fixes.md` (F34); `dihs/final papers/GEODE-DIH-S-paper.md` §6; `dihs/lab notes/README.md` (rule 2)
C4	`hmoe/research/results/tables.md` (P1-T5, P1-T6); `hmoe/research/FINDINGS_AND_THEORY.md` §1.1–1.2; `hmoe/completed/PAPER_1_matched_cost.md` (§3, §4, budget note); `funnel/report/report.md` §12.2; `funnel/experiments.csv`; `hmoe/research/REF9k.log`, `hmoe/research/D128_long.log`
C5	`hmoe/research/results/tables.md` (oracle bound block); `hmoe/research/FINDINGS_AND_THEORY.md` §3; `hmoe/completed/PAPER_3_scale_recursive_routing.md` T4 + verdict §7
C6	`funnel/report/report.md` §10–§11 (59-run ablation table + verdict); `funnel/experiments.csv`; `funnel/hmoe/backfill_routing_ablations.py`, `report_routing_load.py`, `active.py`; `funnel/fix.md`
C7	`funnel/report/report.md` §12.4–§12.7; `funnel/hmoe/generalize.py`
C8	`hmoe/completed/PAPER_3_scale_recursive_routing.md` (abstract, T11, verdict §7); `hmoe/research/FR*_s1.log` (replicate wall-clocks); `hmoe/completed/README.md`
C9	`dihs/sim/results/s33_insample_check.csv`; `dihs/lab notes/07-gates-and-readout-fixes.md` (F35,

Control	Primary artifacts
	F36); `dihs/final papers/GEODE-DIH-S-paper.md` (abstract point 2, §4.6)
C10	`dihs/sim/results/s40_delay_isolated.csv`; `dihs/lab notes/07-gates-and-readout-fixes.md` (F40, F41); `dihs/final papers/GEODE-DIH-S-paper.md` §10; `dihs/lab notes/test-log.csv`; `dihs/lab notes/02-math-audit.md` (F23)

Reproducing this document. Four scripts sit alongside `PROTOCOL.md` and are part of its claim discipline. `make_figures.py` draws all eleven figures from the quoted values — no source-project figure is reused and no new result is plotted. `verify_protocol.py` checks the heading hierarchy, that every embedded figure exists, and that 91 named case-study strings appear in the artifacts they are cited to. `audit_provenance.py` scans every numeric token in the paper against the citable record, recomputes the 17 aggregate cost figures from run logs and result stores, and runs claim-scoped fabrication tests. `render_pdf.py` produces `PROTOCOL.pdf` via pandoc + WeasyPrint.

Both audits are instances of the controls they police, including on themselves. An early version of the orphan scan reported zero problems while matching numbers against a 78 MB corpus in which any float appears somewhere — it was asserting on a quantity, not an effect (C10), and only became a check once scoped to the artifact each claim cites. The figure layout checker failed the same way on first writing: it tested text-on-text collisions, reported everything clean, and missed labels overflowing their own boxes; it now pairs every box with the text drawn inside it and fails if any label spills outside its container.